INDIAN INSTITUTE OF INFORMATION TECHNOLOGY KOTTAYAM

ITW III – Course Project Report

TaskFlow

A Task Management REST API using FastAPI and MySQL with JWT Authentication and Analytics

Submitted by	[Your full name]
Roll number	[Your roll number]
Department	[CSE / CYB / AI-DS]
Course	ITW III (Batch 2)
Course instructor	Dr P. Victer Paul
Month and year	October 2026

Contents

1. Introduction
2. Problem Statement
3. Objectives
4. Technologies Used
5. System Design
6. Methodology
7. Implementation
8. Results and Testing
9. Conclusion
10. Limitations and Future Work
11. References

1. Introduction

Most modern applications, such as mobile apps, websites and dashboards, do not talk to a database directly. They talk to a **web API**: a program that receives a request (for example, "give me my tasks"), checks who is asking, reads or changes the data in a database, and sends back an answer. An API that follows the standard rules of the web (URLs, HTTP methods and JSON data) is called a **REST API**.

This project, **TaskFlow**, is a task management REST API built with Python. Users can register, log in, and manage their own to-do tasks. The data is stored in a **MySQL** relational database. The project is implemented and demonstrated inside a Jupyter Notebook: the notebook defines the database, starts the API server on localhost, fills the database with generated data, tests every endpoint, and finally draws analytics charts.

The project is based on an existing open-source FastAPI task manager (Python-CRUD by isaac0yen) that uses SQLite and has a single table and no login. TaskFlow keeps the same overall structure (models, routes, database setup) but changes the database to MySQL and adds a second table, secure login, filtering and pagination, analytics, data seeding and automated tests.

2. Problem Statement

Simple CRUD (Create, Read, Update, Delete) examples found online are usually small: they use one table, store data in a local file, have no login, and cannot answer questions about the data. A real application needs more:

- Several users must use the same system, but **each user must only see and change their own tasks**.
- Passwords must never be stored as plain text.
- With hundreds of tasks, users need **search, filters and pagination** instead of one long list.
- The data must be stored in a proper database server with related tables.
- Users and managers want **summary information**, such as how many tasks are overdue and how many are completed in each category.

Problem: Design and implement a secure, multi-user task management API backed by a MySQL database that supports full CRUD operations, filtering, search, pagination and analytics, and verify that it works correctly using a realistic amount of data.

3. Objectives

- Build a REST API with FastAPI that runs on localhost and exposes endpoints for tasks and users.
- Store data in **MySQL** using the SQLAlchemy ORM, with two related tables (users and tasks) joined by a foreign key.
- Add **JWT authentication** and **bcrypt password hashing** so that users only access their own data.
- Validate all incoming data with Pydantic and return correct HTTP status codes for errors.
- Support **search, filtering and pagination** of tasks.

- Provide **analytics endpoints** (overdue tasks, completion rate by category, tasks per user) and visualise them with Pandas and Matplotlib.
- Generate about 900 tasks of test data with Faker and test every endpoint from the notebook.

4. Technologies Used

The versions below are the ones used when the final notebook was run.

Tool / Library	Version	Purpose in this project
Python	3.14	Programming language
MySQL Server	8.0.46	Relational database that stores users and tasks
FastAPI	0.141.1	Web framework: turns Python functions into API endpoints, creates automatic Swagger documentation
Uvicorn	0.52.1	Web server that runs the FastAPI application on localhost
SQLAlchemy	2.0.52	ORM: lets tables be used as Python classes, builds the SQL queries
PyMySQL	1.2.3	Driver that connects Python to MySQL
Pydantic	2.13.4	Checks that incoming data has the correct type and allowed values
python-jose	3.5.0	Creates and verifies JWT login tokens
passlib + bcrypt	1.7.4 / 4.0.1	Hashes passwords safely
Faker	40.43.0	Generates realistic fake users, emails and sentences
Requests	2.34.2	Sends HTTP requests to the API (used for testing)
Pandas	3.0.6	Tables for analytics results
Matplotlib	3.11.2	Charts
Jupyter Notebook	-	Environment where the project is implemented and demonstrated

5. System Design

5.1 Architecture

The system has four layers. A client (the notebook or the Swagger page) sends an HTTP request. FastAPI receives it, validates the data and checks the login token. The SQLAlchemy ORM converts the Python objects into SQL, and MySQL stores the data. The answer travels back the same way as JSON.

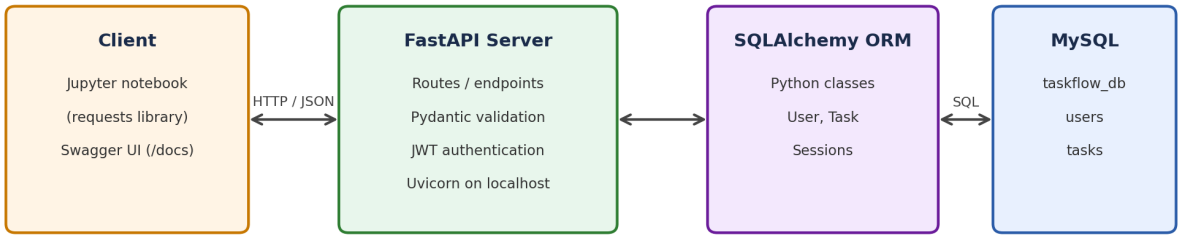


Figure 1: Architecture of TaskFlow

5.2 Database design

There are two tables. One user can own many tasks (a **one-to-many** relationship). The `user_id` column in `tasks` is a **foreign key** that points to `users.id`, so MySQL guarantees that every task belongs to a real user. Columns that are frequently used for searching and filtering (status, priority, category, due_date, user_id) have **indexes**, which let MySQL find rows quickly without scanning the whole table.

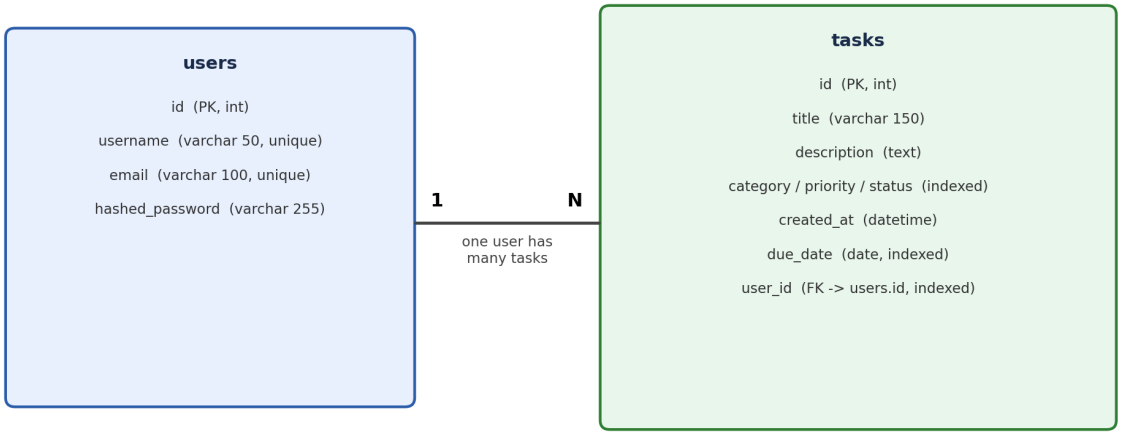


Figure 2: Database tables and their relationship

5.3 API endpoints

Method	Address	Description	Login
POST	/register	Create a new user account	No
POST	/login	Check username and password, return a JWT token	No
POST	/tasks	Create a task	Yes
GET	/tasks	List own tasks with filters (status, priority, category, due_before), title search and pagination (page, page_size)	Yes
GET	/tasks/{id}	Read one task	Yes
PUT	/tasks/{id}	Update some or all fields of a task	Yes
DELETE	/tasks/{id}	Delete a task	Yes
GET	/analytics/overdue	Count and list of overdue, unfinished tasks	Yes

Method	Address	Description	Login
GET	/analytics/completion-by-category	Total, done and completion % per category	Yes
GET	/analytics/tasks-per-user	Number of tasks for every user	Yes

6. Methodology

The project was developed in the following steps:

1. **Study of the base project.** The open-source FastAPI task manager was studied to understand its structure: database setup, task model, and the routes for create, read, update and delete.
2. **Migration from SQLite to MySQL.** The connection string was changed to use MySQL through PyMySQL, and column lengths were added to string columns because MySQL requires a length for VARCHAR.
3. **Database redesign.** A *users* table was added and linked to *tasks* with a foreign key. Indexes were added to the columns used in filters.
4. **Data validation.** Pydantic schemas were written for every request and response, with fixed lists of allowed values for category, priority and status.
5. **Security.** Passwords are hashed with bcrypt. After login the server returns a signed JWT token that must be sent with every protected request.
6. **Endpoints.** CRUD routes were written so that every query is limited to the logged-in user. Filtering, search and pagination were added to the list route.
7. **Analytics.** Three endpoints were written using SQL grouping (GROUP BY) and a table join (JOIN).
8. **Seeding.** A seed script uses Faker to create users and tasks, so the system can be tested with realistic volumes of data.
9. **Testing and visualisation.** The server is started in a background thread inside the notebook, every endpoint is called with the Requests library, and the analytics results are drawn as charts.

6.1 What came from the base project and what was added

Part	Base project	TaskFlow (this project)
Database	SQLite file	MySQL server through PyMySQL
Tables	One tasks table	Two tables (users, tasks) with foreign key and indexes
Login	None	JWT tokens and bcrypt password hashing
Access control	Everyone sees all tasks	Each user only sees their own tasks
Task list	Basic filters (due date, completion)	Filters, title search and pagination
Analytics	None	Three analytics endpoints and charts
Test data	Entered by hand	About 900 tasks generated with Faker

Part	Base project	TaskFlow (this project)
Testing	None	Automated endpoint tests inside the notebook

7. Implementation

All code is in the notebook *TaskFlow_Project.ipynb*. The main parts are explained below with short extracts.

7.1 Database connection

SQLAlchemy creates an *engine* (the connection to MySQL) and a *session factory*. The function `get_db()` gives every request its own database session and closes it afterwards.

```
DATABASE_URL = f"mysql+pymysql://{DB_USER}:{DB_PASSWORD}@{DB_HOST}:{DB_PORT}/{DB_NAME}"
engine = create_engine(DATABASE_URL, pool_pre_ping=True)
SessionLocal = sessionmaker(bind=engine, autoflush=False, autocommit=False)

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

7.2 Models (tables)

Each class becomes one table. The `relationship()` lines let Python move between a user and their tasks.

```
class User(Base):
    __tablename__ = "users"
    id = Column(Integer, primary_key=True, index=True)
    username = Column(String(50), unique=True, nullable=False, index=True)
    email = Column(String(100), unique=True, nullable=False)
    hashed_password = Column(String(255), nullable=False)
    tasks = relationship("Task", back_populates="owner", cascade="all, delete-orphan")

class Task(Base):
    __tablename__ = "tasks"
    id = Column(Integer, primary_key=True, index=True)
    title = Column(String(150), nullable=False)
    category = Column(String(20), nullable=False, index=True)
    priority = Column(String(10), nullable=False, index=True)
    status = Column(String(15), nullable=False, default="Pending", index=True)
    due_date = Column(Date, nullable=False, index=True)
    user_id = Column(Integer, ForeignKey("users.id"), nullable=False, index=True)
    owner = relationship("User", back_populates="tasks")
```

7.3 Validation with Pydantic

Allowed values are fixed with `Literal`. If a client sends a priority that is not Low, Medium or High, FastAPI rejects the request automatically with status code 422, without any extra code.

```
CategoryType = Literal["Study", "Work", "Personal", "Health", "Finance"]
PriorityType = Literal["Low", "Medium", "High"]
StatusType   = Literal["Pending", "In Progress", "Done"]

class TaskCreate(BaseModel):
    title: str = Field(min_length=1, max_length=150)
    category: CategoryType
    priority: PriorityType = "Medium"
    due_date: date
```

7.4 Security: password hashing and JWT

A password is never stored. Only its bcrypt *hash* is saved, and at login the typed password is hashed and compared. After a successful login the server returns a **JWT**, a signed token that contains the username and an expiry time (60 minutes). The client sends it in the header `Authorization: Bearer <token>`. The dependency `get_current_user` reads the token and finds the user; every protected endpoint uses it.

```
def create_token(username):
    expire = datetime.now(timezone.utc) + timedelta(minutes=TOKEN_MINUTES)
    return jwt.encode({"sub": username, "exp": expire}, SECRET_KEY, algorithm=ALGORITHM)

def get_current_user(token=Depends(oauth2_scheme), db=Depends(get_db)):
    username = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM]).get("sub")
    user = db.query(User).filter(User.username == username).first()
    if user is None:
        raise HTTPException(status_code=401, detail="Invalid or expired token")
    return user
```

7.5 Task endpoints with filtering and pagination

Every query starts with `Task.user_id == user.id`, so a user can never reach another user's tasks. Optional filters are added only when the client sends them. Pagination uses `offset` and `limit`: page 2 with page_size 5 skips the first five tasks and returns the next five.

```
query = db.query(Task).filter(Task.user_id == user.id)       # only my tasks
if status:   query = query.filter(Task.status == status)
if priority: query = query.filter(Task.priority == priority)
if search:   query = query.filter(Task.title.like(f"%{search}%"))

total = query.count()
items = (query.order_by(Task.due_date, Task.id)
              .offset((page - 1) * page_size).limit(page_size).all())
```

7.6 Analytics endpoints

The analytics use SQL aggregation. `GROUP BY` with `COUNT` and `SUM` gives the completion rate per category, and an outer join between the users and tasks tables gives the number of tasks of every user.

```
rows = (db.query(User.username, func.count(Task.id))
          .outerjoin(Task, Task.user_id == User.id)
          .group_by(User.id, User.username)
          .order_by(func.count(Task.id).desc()).all())
```

7.7 Seeding the database

The seed script creates one demo user (*demo*) and 15 Faker users. Each user receives about 60 tasks with a random category, priority, status, creation date and due date, so that some tasks are overdue and some are completed. A fixed random seed (42) makes the data repeatable. In the final run the database contained **16 users and 923 tasks** after seeding.

7.8 Running the server inside the notebook

Uvicorn is started in a background thread so the notebook stays usable. Subsequent cells then call the API at `http://127.0.0.1:8000`, exactly as an external client would. The interactive Swagger page is available at `/docs`.

```
config = uvicorn.Config(app, host="127.0.0.1", port=8000, log_level="warning")
server = uvicorn.Server(config)
threading.Thread(target=server.run, daemon=True).start()
```

8. Results and Testing

8.1 Environment

The notebook was run from start to finish without errors on MySQL 8.0.46 with Python 3.14. After seeding, the database contained 16 users and 923 tasks. The demo user owned 65 tasks.

8.2 Endpoint tests

Each test sends a real HTTP request to the running server and checks the status code. All tests behaved as expected.

Test	Request	Expected	Result
Register new user	POST /register	201 Created	201 (user id 17)
Register duplicate user	POST /register	400 Bad Request	400, message: already registered
Login with correct password	POST /login	200 + token	200, JWT token returned
Login with wrong password	POST /login	401 Unauthorized	401, incorrect username or password
List tasks without token	GET /tasks	401	401, not authenticated
List tasks with token	GET /tasks	200	200, 65 tasks for demo user
Create a task	POST /tasks	201	201, task id 924 created
Read the task	GET /tasks/924	200	200
Update the task	PUT /tasks/924	200	200, status changed to In Progress
Delete the task	DELETE /tasks/924	200	200, message: Task 924 deleted
Read the deleted task	GET /tasks/924	404	404, task not found
Invalid priority value	POST /tasks	422	422, input should be Low, Medium or High

Test	Request	Expected	Result
Another user reads demo's task	GET /tasks/{id}	404	404, ownership protected

8.3 Search, filter and pagination

Query	Result
priority = High and status = Pending	6 tasks
category = Study and due within 5 days	11 tasks
title contains "report"	3 tasks
page = 1, page_size = 5 (65 tasks in total)	5 tasks, sorted by due date (first due 2026-08-24)
page = 2, page_size = 5	next 5 tasks (first due 2026-09-14)

8.4 Analytics results

Overdue tasks. The demo user had **20 overdue tasks** (due date in the past and not done). The analytics endpoint lists the ten oldest:

Task id	Title	Priority	Due date
25	Check bank statement	Medium	2026-08-24
51	Buy groceries	Medium	2026-08-30
53	Complete assignment	Low	2026-08-30
63	Review pull request	High	2026-09-06
5	Watch lecture recording	Medium	2026-09-13
36	Revise Python notes	Medium	2026-09-14
21	Complete assignment	Medium	2026-09-15
59	Fix reported bug	Medium	2026-09-19
19	Team meeting notes	Medium	2026-09-21
12	Read research paper	Low	2026-09-23

Completion rate by category (demo user):

Category	Total tasks	Done	Completion rate
Study	17	3	17.6 %
Finance	13	7	53.8 %
Work	15	7	46.7 %
Personal	15	8	53.3 %
Health	5	3	60.0 %

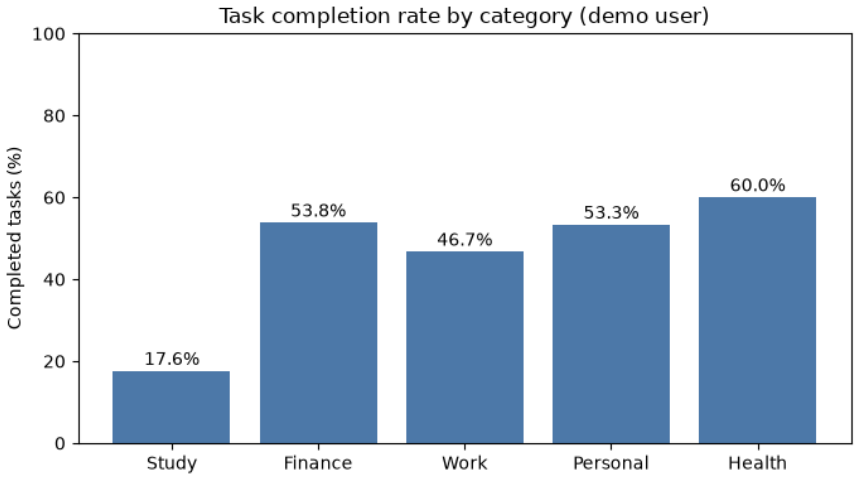


Figure 3: Task completion rate by category (demo user)

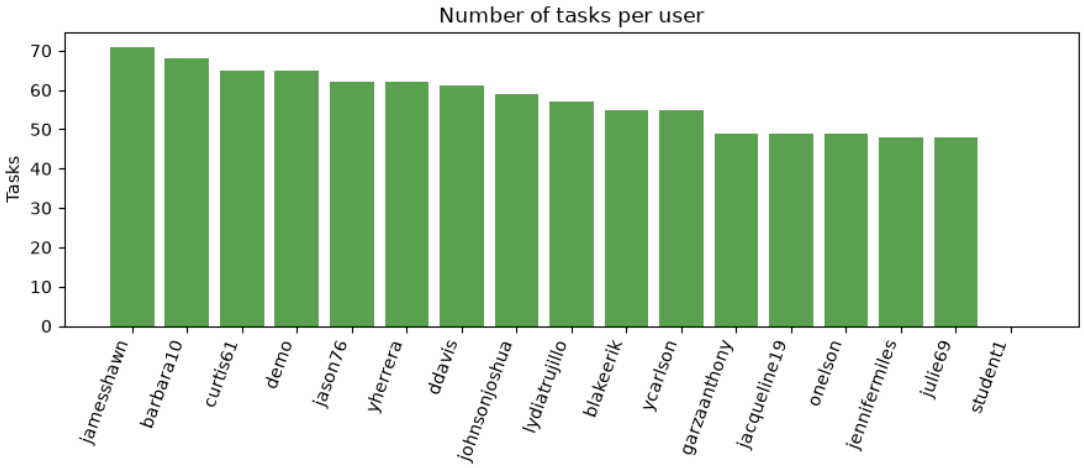


Figure 4: Number of tasks per user (result of a JOIN between the users and tasks tables)

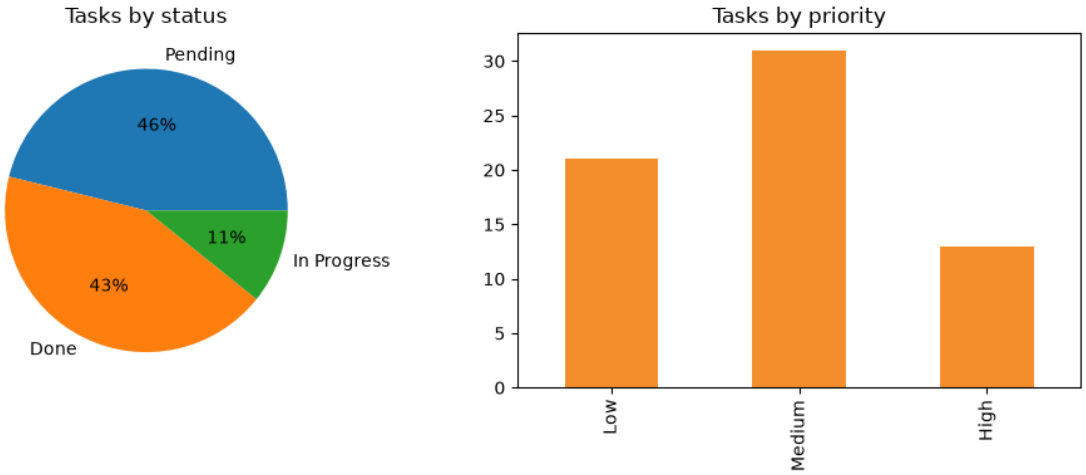


Figure 5: Tasks of the demo user by status and by priority

The charts show that the Study category has the lowest completion rate, so a user would learn from the analytics that study tasks are being postponed more than other tasks. All users received a similar number of tasks, which is expected because the data was generated with the same settings for every user.

9. Conclusion

TaskFlow shows how a small but complete backend system can be built with Python. A FastAPI application serves a MySQL database through the SQLAlchemy ORM; two related tables, a foreign key and indexes give a realistic database design; JWT tokens and bcrypt hashing protect each user's data; Pydantic validation rejects bad input; and SQL aggregation and joins provide useful analytics. The whole system was tested with about 900 generated tasks, and every endpoint returned the expected result, including the error cases.

Compared with the base project, the main improvements are the migration to MySQL, the second table, authentication, access control, search and pagination, analytics, seeding and testing. The project covers the course topics of database integration, data analytics and a Python application that runs as a service.

10. Limitations and Future Work

- The data is **synthetic** (generated by Faker), not real user data. The database starts empty and fills only through the API or the seed script.
- Search supports only the task title.
- The secret key for the tokens is written in the code; a real deployment would read it from an environment variable.
- There is no graphical front end. Swagger UI at /docs and the notebook act as the interface. A web page or a Streamlit app could be added on top of the API.
- Database migrations (Alembic) and formal unit tests with pytest could be added.
- Further features: task reminders, shared tasks between users, and a refresh-token system for longer sessions.

11. References

- Base project: isaac0yen, *Python-CRUD*, GitHub. https://github.com/isaac0yen/Python-CRUD
- FastAPI documentation. https://fastapi.tiangolo.com
- SQLAlchemy documentation. https://docs.sqlalchemy.org
- MySQL Reference Manual. https://dev.mysql.com/doc/
- Pydantic documentation. https://docs.pydantic.dev
- Faker documentation. https://faker.readthedocs.io
- Task descriptions dataset (optional source of task titles): samder03/Project1, Hugging Face. https://huggingface.co/datasets/samder03/Project1